SREBench-API: Comprehensive 20-Scenario Benchmark Specification

Benchmark Name: SREBench-API (Site Reliability & Exploitation Benchmark for APIs)
Proposed in: *PenAgent: An Autonomous LangChain Agent for Adaptive API Penetration Testing*
Category: Multi-Service Cross-Chain API Vulnerabilities
Scope: Distributed Microservices with Trust Relationships

1. Executive Summary & Design Rationale

Existing API security evaluation suites (such as **OWASP Juice Shop** or **DVWS**) test targets in isolation. While effective for single-endpoint issues (e.g. basic SQL injection or individual broken access control), they fail to reflect real-world cloud-native architectures where:

1. Systems are composed of **multiple microservices** (Auth, Billing, Vault, Users, Gateway).
2. Internal microservices **implicitly trust upstream services** or rely on unverified header forwarding.
3. Vulnerabilities **do not exist in any single API call**; they only materialize when requests are orchestrated across services in a specific sequence.

SREBench-API introduces **20 cross-service chaining scenarios** specifically designed to evaluate the unique capability of **PenAgent**: closed-loop, adaptive, multi-step exploit chain discovery.

2. Microservice Architecture & Trust Topology

3. Catalog of the 20 Cross-Service Scenarios

The 20 scenarios are structured across 4 primary architectural patterns:

ID	Scenario Title	Microservices Involved	Chain Depth	CVSS v3.1	Primary OWASP API Category
SRE-01	Cross-Service Trust Header Injection & Secret Leak	Auth → → User → → Vault	3	**8.6** (High)	API1: BOLA / Trust Flaw
SRE-02	Delegated Superadmin Vault Document Extraction	Auth → → User → → Vault	3	**9.4** (Critical)	API5: Broken Function Level Auth
SRE-03	Asynchronous Event Queue Poisoning & Balance Credit	User → → Queue Worker → → Billing	3	**8.2** (High)	API3: Mass Assignment / State Inversion
SRE-04	Upstream Gateway Path Normalization Admin Spoof	Gateway → → Auth → → Admin Proxy	4	**9.1** (Critical)	API7: Server-Side Request Forgery / Gateway
SRE-05	Confused Deputy mTLS Delegation Bypass	Client → → Aggregator → → Data Core	3	**8.8** (High)	API1: BOLA / Identity Stripping
SRE-06	Ephemeral Pre-Signed URL Scope Escalation	Metadata → → Token Mint → → Object Storage	3	**7.9** (High)	API1: Object Level Authorization
SRE-07	Cross-Tenant Webhook Re-Association & Event Tap	Webhook → → Tenant → → Event Stream	4	**8.5** (High)	API1: Cross-Tenant Data Leak
SRE-08	Dual-Token Audience Confusion in Token Exchange	OAuth IdP → → Exchange API → → Resource	3	**8.7** (High)	API2: Broken Authentication
SRE-09	Idempotency Key Desynchronization Charge Hijack	Order → → Payment Gate → → Fulfillment	4	**8.4** (High)	API6: Unrestricted Business Flow
SRE-10	Cascading Soft-Delete Pointer Resurrection	Account → → Relational Cache → → Documents	3	**7.4** (High)	API9: Improper Inventory / Zombie Object
SRE-11	Two-Phase Commit State Decoupling (TOCTOU)	Inventory → → Cart → → Payment Engine	4	**7.7** (High)	API6: Business Flow Race Condition
SRE-12	Asymmetric Cryptographic Handshake Key Downgrade	KMS → → Interconnect → → Policy Agent	3	**9.3** (Critical)	API2: Broken Authentication / Crypto

ID	Scenario Title	Microservices Involved	Chain Depth	CVSS v3.1	Primary OWASP API Category
SRE-13	Chained Telemetry SSRF via Service Discovery Tag	Telemetry → → Discovery (Consul) → → Cloud Meta	4	**9.0** (Critical)	API7: Server-Side Request Forgery
SRE-14	Audit Trail Suppression via Backpressure Drop	Transaction → → Queue Forwarder → → Audit Store	3	**7.1** (High)	API8: Security Misconfiguration / Audit Fail
SRE-15	Cross-Service Cache-Control Public Header Leak	Reverse Proxy → → Aggregator → → User Profile	3	**8.3** (High)	API8: Sensitive Data Exposure
SRE-16	Saga Orchestrator Compensating Transaction Exploit	Workflow Engine → → Provisioner → → IAM Service	4	**9.5** (Critical)	API5: Broken Function Level Auth
SRE-17	GraphQL Schema Stitching Permission Smuggling	GraphQL GW → → Subgraph Catalog → → Orders	3	**8.6** (High)	API5: Authorization Smuggling
SRE-18	Zero-Trust Sidecar Header Injection (`X-Envoy`)	Ingress → → Mesh Sidecar → → Core Banking	3	**9.2** (Critical)	API2: Broken Auth / Perimeter Bypass
SRE-19	Feature-Flag Desynchronization Dark Launch Access	Config Service → → Session Manager → → Beta API	3	**7.8** (High)	API9: Improper Inventory Management
SRE-20	Distributed Sharded Rate-Limit Lockout Evasion	Rate Limiter → → Auth Gate → → User DB	4	**7.5** (High)	API4: Unrestricted Resource Consumption

4. In-Depth Technical Breakdown of Highlighted Scenarios

Scenario SRE-01: Cross-Service Trust Header Injection & Secret Leak

- **Services Involved:** `Auth-Service` (8001), `User-Service` (8002), `Vault-Service` (8003).
- **Vulnerability Mechanism:**
  1. `Auth-Service` issues a valid low-privilege JWT for user `usr_101` (Alice).
  2. Alice queries `GET /users/v1/profile/usr_102` (Bob). Because `User-Service` lacks cross-tenant separation, it leaks Bob's internal tenant identifier (`org_alpha`) and associated resource IDs (`doc_002`).

3. `Vault-Service` checks authorization by reading the incoming `X-Internal-Service` header. If set to `auth-service-internal`, it bypasses cryptographic signature verification under the assumption that the request came over an internal private VPC link.
4. PenAgent discovers Bob's document ID from step 2, mutates its probe by injecting `X-Internal-Service: auth-service-internal`, and downloads Bob's confidential RSA private key (`RSA-SECRET-99412`).

- **Ground-Truth Sequence:**

```
text
Step 1: POST /auth/v1/exchange-token  -> Returns session token
Step 2: GET /users/v1/profile/usr_102 -> Extracts Bob's doc ID (doc_002)
Step 3: GET /vault/v1/documents/doc_002 [Header: X-Internal-Service: auth-service-
internal] -> Secret Exfiltrated
```

Scenario SRE-02: Delegated Superadmin Vault Document Extraction

- **Services Involved:** `Auth-Service`, `User-Service`, `Vault-Service`.
- **Vulnerability Mechanism:**
  1. `User-Service` allows enumerating user metadata (`usr_999` is revealed as Superadmin).
  2. `Vault-Service` implements delegated authorization via `X-Forwarded-User`.
  3. Because the API Gateway fails to strip client-supplied `X-Forwarded-User` headers on internal routes, sending `X-Forwarded-User: usr_999` directly to `GET /vault/v1/documents/doc_admin` grants root privilege and extracts `SYS-MASTER-KEY-888`.
- **CVSS v3.1:** `9.4` (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N).

Scenario SRE-03: Asynchronous Event Queue Poisoning via State Inversion

- **Services Involved:** `User-Service`, `Queue Worker`, `Billing-Service`.
- **Vulnerability Mechanism:**
  1. PenAgent sends a `PUT /users/v1/profile` with an unvalidated custom property: `{"billing_tier": "enterprise_sponsored", "credit_delta": 50000}`.
  2. `User-Service` serializes the raw JSON dictionary into an internal message broker (RabbitMQ/Kafka).
  3. The asynchronous `Billing-Service` background worker ingests the event and updates the user's ledger without verifying cryptographic provenance or running business rule

checks, granting unearned cloud credits.

Scenario SRE-12: Asymmetric Cryptographic Handshake Key Downgrade

- **Services Involved:** `Key Management Service (KMS)`, `Interconnect Proxy`, `Policy Agent`.
- **Vulnerability Mechanism:**
  1. The API allows inter-service token validation using JWKS endpoints.
  2. By exploiting a parameter injection in `GET /auth/v1/jwks?format=pkcs1`, the agent discovers the public key PEM.
  3. PenAgent then signs an arbitrary admin token using the public key as an HMAC secret (the classical `HS256` confusion against an expected `RS256`), which the target microservice accepts because the downstream policy engine checks `alg` from the token header without enforcing an algorithm whitelist.

Scenario SRE-16: Saga Orchestrator Compensating Transaction Hijack

- **Services Involved:** `Workflow Engine`, `Provisioning Worker`, `IAM Service`.
- **Vulnerability Mechanism:**
  1. A distributed Saga pattern manages enterprise user onboarding across 4 sequential steps.
  2. If step 3 (credit card verification) fails, the orchestrator triggers compensating transactions (rollback).
  3. Due to an inversion in the compensation logic, rolling back an aborted user creation executes an un-scoped `IAM:ResetToDefaultPolicy` that inadvertently defaults permissions to the legacy `AdminFallbackRole`.

5. Evaluation Harness & Metrics Mapping

The ground-truth exploit chains for all 20 scenarios are evaluated using the metrics established in **Table 6** of the paper:

$$\text{True Positive Rate (TPR)} = \frac{\text{Confirmed Exploit Chains Detected}}{\text{Total Ground-Truth Vulnerabilities (20)}} \times 100\%$$

True Positive Rate (TPR) = Total Ground-Truth Vulnerabilities (20)Confirmed Exploit Chains Detected × 100%

$$\text{Exploit Chain Depth} = \frac{1}{N} \sum_{i=1}^{N} \text{Number of Chained API Calls in Finding } i$$

Exploit Chain Depth = N1 i=1∑N Number of Chained API Calls in Finding i

$$\text{CVSS Accuracy} = 100\% - \left(\frac{1}{N}\sum_{i=1}^{N} \left|\text{Score}_{\text{agent}} - \text{Score}_{\text{ground_truth}}\right| \times 10\right)$$

CVSS Accuracy = 100% − (N1 i=1∑N |Scoreagent − Scoreground_truth| × 10)

6. Accessing SREBench-API in Your Workspace Container

All 20 scenarios are packaged and runnable inside the `penagent-target-env:latest` Docker image:

- **Scenarios Catalog Endpoint:**

  ```
  bash
  curl http://localhost:8000/scenarios/list | jq .
  ```

- **OpenAPI 3.0 Specification:** Located at `specs/srebench_openapi.json`
- **Ground-Truth Exploit Chains:** Located at `datasets/ground_truth_chains.json`
- **Evaluation Harness Execution:**

  ```
  bash
  python3 datasets/evaluate.py reports/penagent_audit_report.json
  ```